

Módulo 6 – Capítulo 04 – Sección 01

Por qué el chunking determina la calidad de la recuperación

Si el Capítulo 2 estableció que la elección del modelo de embedding determina el techo de calidad del retriever, el Capítulo 4 establece algo aún más fundamental: el chunking determina el suelo. El modelo de embedding más sofisticado del mercado no puede recuperar información que fue destruida en la etapa de fragmentación. Un chunk que corta una oración a la mitad, que mezcla el final de un párrafo sobre contraindicaciones con el inicio de uno sobre dosificación, o que agrupa tres temas distintos en un único fragmento de 1500 tokens, produce un vector que no representa bien ninguno de los temas involucrados y que tendrá baja similitud coseno con queries que buscan cualquiera de ellos. La información existe en el corpus, pero el chunking la ha vuelto irrecuperable.

El mecanismo por el que el chunking afecta la calidad del retriever es directo: el modelo de embedding genera **un único vector** para representar el texto completo del chunk. Si ese texto contiene información coherente sobre un único tema, el vector captura bien ese tema y tiene alta similitud coseno con queries relacionadas. Si el texto mezcla tres temas, el vector es una especie de promedio en el espacio latente de los tres temas —cercano a ninguno de los tres— y tiene similitud coseno mediocre con queries específicas de cualquiera de ellos. Este fenómeno, conocido como **context bleeding** cuando información de dos temas distintos se mezcla en un único chunk, es la principal causa por la que sistemas RAG con modelos de embedding de alta calidad tienen Recall@5 sorprendentemente bajo en producción: el problema no está en el embedding sino en la preparación del texto que se embide.

La evidencia empírica del impacto del chunking es consistente en la literatura: investigaciones sobre benchmarks de recuperación del corpus BEIR muestran que la variación en la estrategia y los parámetros de chunking puede producir diferencias de hasta **15–20 puntos porcentuales en Recall@5**, una magnitud mayor que el impacto de cambiar el modelo de embedding de text-embedding-3-small a text-embedding-3-large. Para poner esto en perspectiva: la diferencia entre un chunking deficiente y uno bien diseñado es mayor que la diferencia entre el mejor modelo de embedding comercial y el más básico. Este dato convierte

al chunking en la primera prioridad de optimización en cualquier sistema RAG, no en un detalle de preprocesamiento.

El problema fundamental del chunking es que el "tamaño ideal" del chunk depende de tres factores que varían por sistema: el tipo de contenido del corpus, el tipo de queries que el sistema debe responder, y el número de tokens de contexto que el LLM generador puede usar eficientemente. Un sistema RAG para contratos legales tiene un corpus con párrafos altamente cohesivos por cláusula (cada cláusula es una unidad semántica natural) y queries que típicamente preguntan por cláusulas específicas: el chunking ideal respeta los límites de cláusula. Un sistema RAG para documentación técnica de APIs tiene un corpus con funciones y endpoints como unidades semánticas naturales: el chunking ideal respeta esos límites. Un sistema RAG para artículos periodísticos tiene un corpus de párrafos cortos e independientes: chunks a nivel de párrafo son los más apropiados. No existe un tamaño universal óptimo, y elegir un tamaño arbitrario porque es el default del framework es un error que se paga en calidad de recuperación desde el primer día.

Puntos críticos del chunking

- **Granularidad semántica:** el chunk ideal contiene una unidad de información coherente —un procedimiento, una definición, una cláusula legal— que puede recuperarse de forma independiente sin perder su significado por falta de contexto.
- **Impacto del chunk en el vector embedding:** chunks de más de 512 tokens tienden a producir vectores promediados que capturan temas múltiples; chunks de menos de 50 tokens tienen insuficiente contexto semántico para que el embedding capture significado útil.
- **Context bleeding:** información de dos temas distintos mezclada en un chunk produce un vector de baja calidad que no representa bien ninguno de los dos; el límite de chunk en el punto equivocado es la causa más frecuente de context bleeding.
- **Dependencia del dominio:** el chunking óptimo varía radicalmente según el tipo de contenido; lo que funciona para artículos científicos (chunking por sección) es ineficiente para FAQs (chunking por par pregunta-respuesta) o para código fuente (chunking por función o clase).
- **Impacto en el generador:** chunks demasiado cortos dan al LLM generador contexto insuficiente para responder bien; chunks demasiado largos consumen tokens de contexto innecesarios y elevan el costo de inferencia proporcionalmente.
- **Evaluación empírica obligatoria:** la única forma válida de elegir la estrategia de chunking es medir Recall@K en un dataset de evaluación representativo; no existe un

tamaño universalmente óptimo; 512 tokens con 10% overlap es un punto de partida razonable, raramente el óptimo.

Nota del Arquitecto: He revisado más de veinte implementaciones de RAG en producción y la historia es consistente: el chunking fue el default del framework, nunca fue evaluado con datos reales, y es la primera causa de recall deficiente cuando el sistema llega a producción. El fix requiere reindexar todo el corpus. El tiempo de construir un dataset de evaluación de 100 queries y probar dos estrategias de chunking es de un día; el tiempo de reindexar en producción cuando el corpus tiene 2 millones de documentos es de semanas. La inversión en el diseño del chunking antes del primer índice es siempre la más rentable.

El impacto del chunking en la calidad del sistema RAG es suficientemente grande como para que sea el primer componente a diseñar y evaluar en cualquier proyecto, antes de elegir el modelo de embedding o la base vectorial. La siguiente sección analiza las estrategias de chunking concretas disponibles, sus mecanismos y sus trade-offs para ayudar a tomar esa decisión con criterios técnicos.

Módulo 6 – Capítulo 04 – Sección 02

Estrategias de chunking: fijo, semántico, recursivo y por estructura del documento

No existe una única estrategia de chunking universal. Cada enfoque tiene un perfil de calidad, costo computacional y adecuación a tipos de contenido que lo hace más o menos apropiado para cada caso de uso. Conocer las seis estrategias principales —con sus mecanismos técnicos, sus implementaciones concretas y sus trade-offs— es la base para tomar una decisión informada antes de la primera línea de código de indexación.

El **chunking por tamaño fijo** (`CharacterTextSplitter` en LangChain) divide el texto en segmentos de N caracteres con un overlap de M caracteres, sin ningún análisis del contenido. Es el método más rápido y predecible, pero también el más ciego semánticamente: corta oraciones a la mitad, separa preguntas de sus respuestas, fragmenta definiciones. Solo es apropiado cuando el texto ya está preprocesado para ser altamente homogéneo y donde los cortes arbitrarios tienen baja probabilidad de partir unidades semánticas importantes. En la práctica, es el método que se usa cuando el tiempo de implementación es el único criterio, nunca cuando la calidad del retriever es prioritaria.

El **RecursiveCharacterTextSplitter** (LangChain) es el estándar de facto para texto libre no estructurado y el punto de partida correcto para la mayoría de los sistemas. Su mecanismo es una jerarquía de separadores configurables: el algoritmo intenta primero dividir por el separador de mayor nivel (`"\n\n"`, que corresponde a párrafos), luego por el siguiente (`"\n"`, líneas), luego por punto y espacio (`". "`), y finalmente por espacio si ningún separador produce chunks del tamaño deseado. Este enfoque respeta la estructura natural del texto en la medida en que ésta existe, produciendo chunks que raramente cortan en medio de una oración y frecuentemente respetan los límites de párrafo. Es un gran avance sobre el chunking fijo con una implementación de la misma complejidad; para texto con estructura básica de párrafos, es el método correcto por defecto.

El **chunking semántico** (`SemanticSplitterNodeParser` en LlamaIndex) lleva la idea de respetar la estructura semántica del texto al extremo: en lugar de usar separadores de texto

para inferir los límites, usa el propio modelo de embedding para detectarlos. El algoritmo calcula la similitud coseno entre embeddings de oraciones consecutivas; cuando la similitud cae bruscamente entre la oración N y la N+1, interpreta ese quiebre como un límite semántico natural y coloca el límite de chunk en ese punto. El resultado son chunks con coherencia temática mucho mayor que el `RecursiveCharacterTextSplitter`, pero a un costo significativo: el proceso requiere embedir cada oración del corpus durante la ingesta, multiplicando el costo de embedding por un factor de 5–10x comparado con embedir directamente los chunks finales.

El **chunking por estructura del documento** aprovecha los separadores explícitos que el autor del documento incluyó: headers de Markdown (`#`, `##`, `###`), tags HTML (`<h1>`, `<h2>`, `<section>`), o secciones de LaTeX (`\section`, `\subsection`). `MarkdownHeaderTextSplitter` y `HTMLHeaderTextSplitter` en LangChain extraen la jerarquía de headers del documento y dividen en esos puntos, propagando los headers como metadatos del chunk (por ejemplo, `{"H1": "Guía de Usuario", "H2": "Instalación", "H3": "Requisitos"}`). Esta propagación de metadatos es especialmente valiosa porque permite al sistema filtrar chunks por sección en tiempo de búsqueda y permite al LLM generador atribuir la fuente con precisión. Para corpora con documentos bien estructurados (documentación técnica, artículos científicos, manuales de producto), este es el método de mayor calidad sin el costo del chunking semántico.

El **parent-child chunking** es una estrategia de doble indexación que combina la precisión de la recuperación de chunks pequeños con la riqueza contextual de fragmentos más grandes. Los chunks pequeños (128 tokens) se indexan para recuperación precisa: al ser muy específicos, su vector tiene alta similitud coseno con queries igualmente específicas. Al recuperar, en lugar de devolver el chunk pequeño al LLM, el sistema recupera el chunk padre (512 tokens) que lo contiene, proporcionando el contexto necesario para una respuesta completa. LlamaIndex implementa este patrón como `ParentDocumentRetriever`; LangChain lo implementa en `ParentDocumentRetriever` con `InMemoryStore` o `LocalFileStore` para mantener la referencia entre chunks hijos y padres.

El **chunking por proposiciones** es la técnica de mayor calidad y mayor costo: usa un LLM para segmentar el texto en proposiciones atómicas —unidades de conocimiento mínimas que expresan un único hecho— e indexa cada proposición como chunk independiente. Una proposición es algo como "El medicamento X tiene una vida media de 12 horas" o "El artículo 5.3 del contrato establece una penalización del 2% mensual por retraso". La precisión de recuperación con proposiciones es máxima porque cada vector representa exactamente un

hecho, pero el costo de generar las proposiciones con un LLM durante la ingesta es prohibitivo para corpus grandes (aproximadamente \$0.001 por documento con GPT-4o-mini, lo que representa \$1000 para un corpus de 1 millón de documentos solo en el paso de proposicionalización).

Estrategias de chunking con sus trade-offs

- **Chunking por tamaño fijo (CharacterTextSplitter):** $O(N)$ en tiempo, sin overhead de análisis; produce cortes arbitrarios; adecuado únicamente cuando el texto es muy homogéneo y ya está preprocesado; nunca es el método óptimo.
- **RecursiveCharacterTextSplitter:** jerarquía de separadores configurable; respeta estructura natural del texto; estándar de facto para texto libre; el punto de partida correcto para la mayoría de los sistemas antes de optimizaciones más sofisticadas.
- **Chunking semántico (SemanticSplitterNodeParser):** detecta límites semánticos con embeddings de oraciones; produce chunks de alta cohesión temática; costo 5–10x mayor en ingesta por las llamadas adicionales al modelo de embedding.
- **Chunking por estructura (MarkdownHeaderTextSplitter):** respeta la jerarquía explícita del documento; propaga headers como metadatos; cero overhead adicional sobre el chunking fijo; ideal para corpora con documentación bien estructurada.
- **Parent-child chunking (ParentDocumentRetriever):** indexa chunks pequeños para precisión, recupera chunks grandes para contexto; combina las ventajas de granularidades distintas; requiere almacenamiento adicional para mantener la referencia padre-hijo.
- **Chunking por proposiciones:** máxima precisión de recuperación; costo de ingesta elevado por las llamadas al LLM; apropiado solo para corpus pequeños donde la precisión factual es crítica (medicina, legal, ciencias).

La elección de la estrategia de chunking debe estar guiada por el tipo de contenido del corpus y la tipología de queries del sistema, validada con el dataset de evaluación interno. Los parámetros de tamaño y overlap dentro de la estrategia elegida son el siguiente nivel de calibración, que la sección siguiente analiza en detalle.

Módulo 6 – Capítulo 04 – Sección 03

Chunk size vs chunk overlap: trade-offs de precisión y cobertura

Una vez elegida la estrategia de chunking, los dos parámetros más influyentes en la calidad de la recuperación son `chunk_size` y `chunk_overlap`. Estos parámetros operan como dimensiones ortogonales de un espacio de trade-offs: `chunk_size` controla la especificidad semántica del vector (el "qué" captura el embedding), y `chunk_overlap` controla la redundancia entre chunks consecutivos (cuánto contexto se preserva en los límites). Ninguna combinación es universalmente óptima: el punto ideal depende del tipo de corpus, del tipo de queries y de cómo el LLM generador usa el contexto.

El **chunk_size** tiene un efecto directo y no lineal en la calidad del vector de embedding. Cuando el chunk es pequeño (64–128 tokens), el embedding captura con alta especificidad el contenido del fragmento: un chunk de una única oración produce un vector que está muy cerca, en el espacio latente, de queries que preguntan exactamente sobre esa oración. La contrapartida es doble: el chunk carece del contexto necesario para que el LLM generador formule una respuesta completa (una sola oración raramente es suficiente para responder incluso preguntas simples), y el número total de chunks en el índice se multiplica, aumentando el costo de almacenamiento y la latencia de búsqueda. Cuando el chunk es grande (1024–2048 tokens), el embedding está "promediando" múltiples párrafos o incluso múltiples temas: el vector resultante es menos específico, tiene menor similitud coseno con queries específicas sobre cualquiera de los subtemas del chunk, y el Recall@K disminuye para queries de precisión alta.

El rango de **512–768 tokens** es el punto de equilibrio más robusto empíricamente para la mayoría de los corpora de texto técnico o documental. Este rango cubre típicamente 1–3 párrafos coherentes, suficiente para que el embedding capture el tema principal con especificidad razonable, y suficiente para que el LLM generador reciba contexto útil para formular una respuesta. Para QA factual donde la respuesta es un dato puntual (una cifra, una fecha, un nombre), el rango de 256–512 tokens puede ser más apropiado porque la especificidad del vector es más importante que la riqueza del contexto. Para síntesis de

documentos complejos donde la respuesta requiere entender el argumento completo de una sección, el rango de 1024–2048 tokens puede ser necesario, aunque a costa de mayor imprecisión en el retriever.

El **chunk_overlap** introduce redundancia controlada en los límites de los chunks. Sin overlap, la información que cae exactamente en el punto de corte entre dos chunks consecutivos puede quedar en el segundo chunk sin el contexto del primero, o en el primero sin la conclusión del segundo. Un overlap del 10–20% del chunk_size (50–150 tokens para un chunk de 512) garantiza que las últimas 50–150 palabras de cada chunk aparecen también al inicio del chunk siguiente, preservando la continuidad del contexto en los límites. El costo es que el número total de chunks en el índice crece en un 10–20%, incrementando el costo de almacenamiento y la latencia de búsqueda en la misma proporción.

Un overlap excesivo (>50% del chunk_size) produce chunks altamente redundantes que inflan el índice y tienen un efecto perverso: cuando el retriever recupera los top-K chunks, varios de ellos pueden ser solapamientos del mismo pasaje del documento original, ocupando slots del contexto con contenido casi idéntico y reduciendo efectivamente el número de fuentes independientes disponibles para el LLM. Este escenario —recuperar 5 chunks que son variaciones del mismo párrafo— es más perjudicial que recuperar 5 chunks independientes de menor similitud coseno. El overlap correcto no es "más es mejor" sino "suficiente para no perder contexto en los límites".

Para corpus con separadores semánticos naturales (headers, fin de cláusula, fin de sección), el overlap puede reducirse a cero: si el chunking respeta los límites semánticos del documento, no hay información que se pierda en los límites. Esta es la ventaja del chunking por estructura sobre el chunking por tamaño fijo: la ausencia de overlap reduce el número de chunks sin perder continuidad semántica.

Trade-offs técnicos de tamaño y overlap

- **Chunk size 128–256 tokens:** alta especificidad del vector; útil para QA factual donde la respuesta es un dato puntual; riesgo de contexto insuficiente para el LLM generador; requiere K mayor para cubrir suficiente contexto en la respuesta.
- **Chunk size 512–768 tokens:** el rango de equilibrio más robusto para texto técnico o documental; cubre 1–3 párrafos; vector suficientemente específico con contexto suficiente para generación; punto de partida empírico correcto para la mayoría de los sistemas.

- **Chunk size 1024–2048 tokens:** adecuado para documentos con alta coherencia temática por sección (contratos por cláusula, artículos científicos por sección); requiere modelos de embedding con ventanas de contexto largas; riesgo de vectores promediados si la sección mezcla subtemas.
- **Chunk overlap 0%:** sin redundancia; correcto cuando los separadores de chunking son semánticamente significativos (headers de sección, fin de cláusula); máxima eficiencia de almacenamiento.
- **Chunk overlap 10–20% (50–150 tokens sobre chunk de 512):** garantiza continuidad de contexto en límites arbitrarios; incrementa el corpus indexado en 10–20%; recomendado como configuración por defecto para RecursiveCharacterTextSplitter.
- **Chunk overlap >50%:** chunks altamente redundantes que inflan el índice; riesgo de recuperar múltiples versiones del mismo pasaje; raramente justificable; puede indicar que la estrategia de chunking no es la adecuada para el tipo de documento.

***Nota del Arquitecto:** La calibración de `chunk_size` y `chunk_overlap` no debería hacerse con un único experimento sino con un grid search sobre el dataset de evaluación. Con `chunk_size` $\in \{256, 512, 768, 1024\}$ y `overlap` $\in \{0\%, 10\%, 20\%\}$, son 12 configuraciones; ejecutar el pipeline de indexación y evaluación para cada una sobre el corpus de prueba cuesta horas de cómputo pero proporciona evidencia cuantitativa para una decisión que impacta la calidad de producción durante meses. En la práctica, la configuración ganadora difiere del default del framework en el 80% de los casos que he evaluado.*

La calibración de `chunk_size` y `chunk_overlap` es el primer experimento de optimización que debe ejecutarse sobre el sistema RAG, con resultados medidos contra el dataset de evaluación interno. Con la estrategia y los parámetros de chunking correctos, la siguiente sección explora cómo enriquecer los chunks con metadatos y contexto adicional para elevar aún más la calidad del índice.

Módulo 6 – Capítulo 04 – Sección 04

Enriquecimiento de chunks: metadatos, resúmenes y jerarquías de contexto

El chunk indexado no necesita ser solo el texto crudo del fragmento: puede —y en sistemas de producción de alta calidad, debe— incluir información adicional que mejore tanto la precisión de la recuperación como la utilidad del contexto para el LLM generador. El enriquecimiento de chunks es una familia de técnicas que transforman un fragmento de texto plano en una unidad de conocimiento más rica: con metadatos que permiten filtrado preciso, con contexto adicional que mejora el embedding, con estructura que facilita la atribución de fuentes. Cada técnica de enriquecimiento añade costo a la etapa de ingesta pero mejora la calidad del índice de forma cuantificable y permanente.

Los **metadatos de procedencia** son la forma más básica y más importante de enriquecimiento. Adjuntar a cada chunk información sobre su origen —nombre del archivo, número de página, título de la sección, fecha de creación, tipo de documento, versión del documento, identificador único del documento padre— tiene dos efectos directos. Primero, habilitan filtrado preciso en tiempo de búsqueda: en lugar de buscar en todo el corpus, el retriever puede restringir la búsqueda a documentos de un tipo específico, dentro de un rango de fechas, o pertenecientes a una categoría particular. Segundo, permiten al LLM generador atribuir las fuentes de forma precisa en su respuesta: "Según el Manual de Producto v3.2, sección 5.4, página 47...". Esta atribución es un requisito de negocio en muchos contextos empresariales (auditoría, cumplimiento, procesos legales) que es imposible sin metadatos adecuados en el índice.

La técnica de **contextual retrieval** publicada por Anthropic en septiembre de 2024 es el avance más significativo en enriquecimiento de chunks de los últimos años. El problema que resuelve es el siguiente: cuando un chunk es extraído de su documento, pierde el contexto que permite entender su rol en el argumento más amplio. Un chunk que dice "La tasa de recurrencia en el grupo de control fue del 23%" es difícilmente recuperable para una query sobre "resultados del ensayo clínico del medicamento X" si el chunk no menciona el nombre del medicamento ni del ensayo. La solución de contextual retrieval es usar un LLM para

generar un breve párrafo de contexto —de 50 a 100 tokens— por cada chunk, que describe el rol del fragmento dentro del documento completo. Este contexto se prepone al chunk antes de generar el embedding, de forma que el vector resultante captura no solo el contenido del chunk sino también su relevancia en el contexto del documento. Los experimentos publicados por Anthropic muestran mejoras de Recall@20 de entre el 35% y el 49% usando esta técnica con `claude-3-haiku-20240307` a un costo adicional de aproximadamente \$1 por millón de tokens procesados —un overhead moderado que en la mayoría de los casos justifica el delta de calidad.

La **indexación jerárquica** (parent-child indexing) aplica el mismo principio del parent-child chunking de la sección anterior al enriquecimiento: indexar chunks pequeños para recuperación precisa (son los que se usan para la búsqueda ANN) pero vincularlos a sus chunks padres o al documento completo para la generación. Cuando el retriever encuentra un chunk hijo relevante, el sistema recupera el chunk padre asociado —que contiene más contexto— y lo envía al LLM en lugar del chunk pequeño. Este patrón combina la ventaja de la recuperación precisa (los chunks pequeños con vectores específicos tienen Recall@K alto para queries específicas) con la ventaja de la generación con contexto rico (los chunks padres con más texto dan al LLM suficiente información para responder de forma completa).

Las **preguntas hipotéticas** (hypothetical questions indexing) son una técnica alternativa al contextual retrieval: en lugar de prepender contexto sobre el documento, se generan para cada chunk 3–5 preguntas que ese chunk podría responder, y se indexan las preguntas en lugar del texto original del chunk. La intuición es que la similitud coseno entre la query del usuario y una pregunta hipotética del mismo tipo es generalmente mayor que la similitud entre la query del usuario y el texto del documento que responde la pregunta (que puede usar un vocabulario y una estructura distintos). Esta técnica es especialmente efectiva para corpora de documentación técnica o legal donde el vocabulario de las queries de los usuarios (informal, coloquial) difiere significativamente del vocabulario del corpus (formal, técnico).

Técnicas de enriquecimiento de chunks

- **Metadatos de procedencia:** `source_file`, `page_number`, `section_title`, `document_type`, `created_date`, `document_id` por chunk; habilitan filtrado en búsqueda y atribución de fuentes en respuestas; el costo es solo el almacenamiento adicional de los campos en el payload de la base vectorial.
- **Contextual retrieval (Anthropic, 2024):** generar con Claude Haiku un párrafo de contexto de 50–100 tokens por chunk que describe su rol en el documento completo;

prepend al texto del chunk antes de embedir; mejora Recall@20 entre 35% y 49% según experimentos de Anthropic; costo adicional de ~\$1 por millón de tokens con claude-3-haiku.

- **Resúmenes de chunk:** generar un resumen de 1–3 oraciones del chunk con un LLM y usarlo como texto alternativo para el embedding; el vector del resumen es más específico para queries cortas; indexar ambos vectores permite dual retrieval (buscar por similitud con el texto completo y por similitud con el resumen).
- **Indexación jerárquica (parent-child):** chunks de 128 tokens para recuperación precisa vinculados a chunks padres de 512 tokens; al recuperar, devolver el chunk padre al LLM; implementado en LlamaIndex como `ParentDocumentRetriever` y en LangChain con `InMemoryStore`.
- **Title prepending:** prepend el título del documento a cada chunk antes de generar el embedding; mejora la recuperación cuando la query menciona el nombre del documento o su tema principal; costo nulo (solo concatenación de texto).
- **Hypothetical questions indexing:** generar 3–5 preguntas hipotéticas por chunk con un LLM y usarlas como texto de indexación; mejora la recuperación cuando el vocabulario de las queries de usuarios difiere del vocabulario del corpus; costo de generación LLM durante la ingesta.

El enriquecimiento de chunks es una inversión en calidad del índice con retorno permanente: el costo adicional de ingesta se paga una vez y el beneficio de mejora del Recall@K se obtiene en cada consulta durante toda la vida del índice. La siguiente sección aborda cómo gestionar los cambios del corpus de forma eficiente, sin reindexar todo cada vez que un documento se actualiza.

Módulo 6 – Capítulo 04 – Sección 05

Indexación incremental: actualizaciones sin reindexar toda la colección

Un corpus de producción nunca es estático: los documentos se actualizan, se añaden nuevos contenidos, se eliminan materiales obsoletos con errores. Para un sistema RAG cuyo corpus tiene 500.000 documentos, la reindexación completa en cada cambio implicaría regenerar el embedding de cada chunk (millones de llamadas al modelo de embedding), reescribir el índice vectorial completo (horas de I/O sobre la base de datos vectorial) y validar la calidad del nuevo índice (ejecutar el pipeline de evaluación). Este proceso puede tardar entre horas y días dependiendo del tamaño del corpus, haciendo operativamente inviable cualquier actualización frecuente del conocimiento. La indexación incremental es la solución: procesar solo los documentos que han cambiado, actualizando únicamente los vectores afectados sin tocar el resto del índice.

El componente central de cualquier sistema de indexación incremental es el **mecanismo de detección de cambios**. La estrategia más simple y más robusta es el content hashing: calcular un hash criptográfico (SHA-256 o MD5) del contenido de cada documento y compararlo con el hash almacenado en una base de datos de tracking (PostgreSQL, DynamoDB, Redis). Si el hash del documento en el sistema de origen coincide con el hash almacenado, el documento no ha cambiado y se omite el procesamiento. Si el hash difiere, el documento fue modificado y requiere reindexación. Si no existe un hash almacenado, el documento es nuevo y requiere indexación inicial. Esta lógica de "hash match → skip, hash mismatch → reindex, hash absent → index" es idempotente: el pipeline puede ejecutarse múltiples veces sobre el mismo corpus sin efectos secundarios, lo que es esencial para poder reintentar automáticamente en caso de fallos.

El **document tracking store** es la base de datos que persiste el estado de indexación de cada documento: `doc_id`, `content_hash`, `indexed_at`, `chunk_ids[]` (los IDs de todos los chunks en el índice vectorial que corresponden a este documento). Esta información tiene un rol operacional crítico durante las actualizaciones: cuando se detecta que un documento ha cambiado, el sistema necesita eliminar todos los chunks antiguos del índice antes de insertar

los nuevos. Sin el `chunk_ids[]` almacenado en el tracking store, sería necesario buscar en el índice vectorial todos los chunks con `doc_id = X` mediante un filtro de metadatos, lo que es más costoso que una consulta directa al tracking store. Alternativamente, si los chunks se identifican con IDs deterministas derivados del `doc_id` y el número de chunk (por ejemplo, `SHA256(doc_id + "_" + chunk_index)`), el tracking store puede calcular los IDs sin necesidad de almacenarlos explícitamente.

La **integración con sistemas de gestión documental** es el mecanismo que dispara el pipeline de indexación cuando un documento cambia. Dos patrones principales: el procesamiento batch programado ejecuta el pipeline periódicamente (cada hora, cada noche) escaneando todo el corpus, calculando hashes y procesando solo los cambios; es el enfoque más simple y adecuado cuando la latencia de actualización de minutos u horas es aceptable. El procesamiento streaming procesa los cambios en tiempo real mediante eventos: sistemas como SharePoint, Confluence, Notion y Google Drive ofrecen webhooks que disparan una notificación cuando un documento es creado, modificado o eliminado; el pipeline de ingesta se suscribe a esos eventos y procesa el documento afectado con latencia de segundos. Apache Kafka, AWS SQS o Google Pub/Sub son los intermediarios de mensajería que desacoplan el sistema de gestión documental del pipeline de ingesta, garantizando durabilidad y reintentos automáticos en caso de fallos transitorios.

La **eliminación atómica y la inserción secuencial** durante las actualizaciones debe seguir un orden específico para evitar estados inconsistentes. El orden correcto es: (1) eliminar del índice vectorial todos los chunks con `doc_id = X` usando un `delete_by_filter`; (2) procesar el documento actualizado (parsear, chunkear, embedir); (3) insertar los nuevos chunks en el índice con `upsert`. El orden inverso —insertar primero, eliminar después— produce un período donde coexisten la versión antigua y la nueva del mismo documento, devolviendo resultados duplicados o contradictorios en las búsquedas durante ese período. Para sistemas con SLO de consistencia estricta, el patrón blue-green a nivel de documento —preparar los nuevos chunks offline, insertarlos con `is_draft=true`, luego en una operación atómica actualizar a `is_draft=false` y eliminar los chunks antiguos— elimina cualquier período de inconsistencia.

Componentes de la indexación incremental

- **Change detection con content hashing:** calcular SHA-256 del contenido de cada documento; comparar con hash almacenado; documentos con hash idéntico se omiten; documentos con hash diferente o ausente se procesan; idempotencia garantizada.

- **Document tracking store:** base de datos (PostgreSQL, DynamoDB) con `doc_id`, `content_hash`, `indexed_at`, `chunk_ids[]`; permite coordinar actualizaciones atómicas y reconstruir qué chunks pertenecen a qué documento sin consultas al índice vectorial.
- **Eliminación → inserción en orden correcto:** `delete_by_filter(doc_id=X)` → procesar documento → `upsert` de nuevos chunks; garantiza que no coexisten versiones antigua y nueva; el período de ausencia del documento del índice (entre `delete` y `upsert`) es típicamente <1 segundo.
- **Pipeline batch vs. streaming:** batch (Airflow, Prefect, cron) para latencia de actualización de minutos a horas; streaming (Kafka, SQS, webhooks) para latencia de segundos; la elección depende de la frecuencia de actualización y del requisito de frescura del índice.
- **Dead letter queue para documentos fallidos:** documentos que fallan repetidamente (parsing error, timeout de embedding, fallo de escritura en índice) se envían a una cola separada con el error y el documento original; requieren revisión manual o estrategia de procesamiento alternativa.
- **Idempotencia del pipeline:** el pipeline debe ser ejecutable múltiples veces sin efectos secundarios; el mecanismo de content hashing garantiza que documentos ya indexados no se reprocesen; operaciones de `upsert` en lugar de `insert` garantizan que reintentos no produzcan duplicados.

Nota del Arquitecto: El diseño incremental desde el inicio es uno de los principios de ingeniería más importantes en RAG y también uno de los más frecuentemente ignorados. "Reindexaremos todo una vez por semana" funciona cuando el corpus tiene 10.000 documentos. Con 500.000 documentos, la reindexación semanal tarda 18 horas y consume el 75% del límite de tasa de la API de embedding. Añadir capacidad incremental a un sistema diseñado para reindexación completa requiere refactorizar el tracking store, el pipeline de ingesta y la lógica de eliminación. El costo de diseñarlo correctamente desde el inicio es de horas; el costo de refactorizarlo en producción es de semanas.

Con la indexación incremental implementada, el corpus del sistema RAG puede mantenerse actualizado de forma continua sin overhead operacional creciente. El capítulo concluye con la reflexión sobre por qué el chunking y la preparación de documentos son la decisión de mayor impacto en la calidad del sistema, antes de pasar al Capítulo 5 que aborda las estrategias de recuperación sobre el índice construido.

Módulo 6 – Capítulo 04 – Sección 06

Cierre: el chunking es la decisión de ingeniería más impactante en RAG

La evidencia empírica acumulada en el campo converge en una conclusión que contradice la intuición inicial de muchos ingenieros que se acercan a RAG por primera vez: el componente más determinante de la calidad de un sistema RAG no es el modelo de lenguaje generador, ni siquiera el modelo de embedding, sino el chunking y la preparación de los documentos. La razón es matemáticamente directa: el embedding solo puede capturar lo que está en el chunk que recibe; si el chunk fue cortado en el lugar equivocado, si mezcla temas distintos, o si carece del contexto necesario para que su significado sea autocontenido, ningún modelo de embedding puede recuperar la calidad perdida en esa etapa. La degradación ocurre upstream y es irrecuperable downstream.

Esta primacía del chunking tiene una implicación práctica inmediata: el tiempo y el rigor de ingeniería deben distribuirse de forma diferente a la intuición. Los equipos tienden a gastar días comparando modelos de LLM generadores y horas en el chunking, cuando la relación de impacto sugiere el orden inverso. Un sistema con chunking bien diseñado, metadatos ricos y enriquecimiento con contextual retrieval puede compensar un modelo de embedding de segunda línea. Un sistema con el mejor modelo de embedding del mercado pero un chunking que corta unidades semánticas arbitrariamente tendrá un Recall@5 persistentemente bajo que ninguna optimización del retriever o del generador puede corregir.

La especificidad de dominio del chunking es el segundo punto crítico de este capítulo. El chunking óptimo para contratos legales —respetando los límites de artículos y subartículos, propagando el número de artículo como metadato— es completamente distinto al óptimo para documentación técnica de APIs —respetando los límites de endpoint o función, preservando la firma como metadato— o para artículos periodísticos —chunking por párrafo con el titular como contexto prepended. Esta especificidad de dominio implica que el chunking no puede ser delegado al framework ni elegido por defecto: debe ser diseñado por alguien con conocimiento del corpus y de cómo los usuarios formulan sus queries, evaluado

con datos reales del dominio y validado con el dataset de evaluación interno antes de la indexación inicial.

El diseño correcto del chunking no es un lujo de sistemas maduros: es el prerequisite para que cualquier otro componente del pipeline pueda funcionar bien. Sin un chunking que preserve las unidades semánticas del corpus, el modelo de embedding no puede producir vectores que capturen el contenido relevante, el retriever no puede encontrar los chunks correctos, y el LLM no puede generar respuestas fundamentadas en evidencia. El chunking correcto es la fundación; todo lo demás se construye sobre ella.

El Capítulo 5 construye sobre esta fundación para analizar las estrategias de recuperación: cómo combinar la búsqueda semántica vectorial con la búsqueda léxica BM25 para obtener lo mejor de ambos enfoques, cómo el reranking mejora la precisión de los chunks presentados al LLM, y cómo la transformación de queries antes de la búsqueda puede recuperar documentos que la búsqueda directa sistemáticamente pierde.

"Garbage in, garbage out. En ningún sistema de recuperación de información esto es más verdad que en la calidad del preprocesamiento: cualquier pérdida de información en la transformación del documento original al fragmento indexado es irrecoverable en tiempo de consulta." — Karen Spärck Jones, pionera de la recuperación de información y creadora del cálculo de TF-IDF.